



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-7 文件系统之底层设备

1. 文件系统类型的文件
2. qemu接入块设备——磁盘
3. 文件系统的结构
4. 文件系统读写
5. 内核VFS初始化
6. 总结

1. 文件系统类型的文件

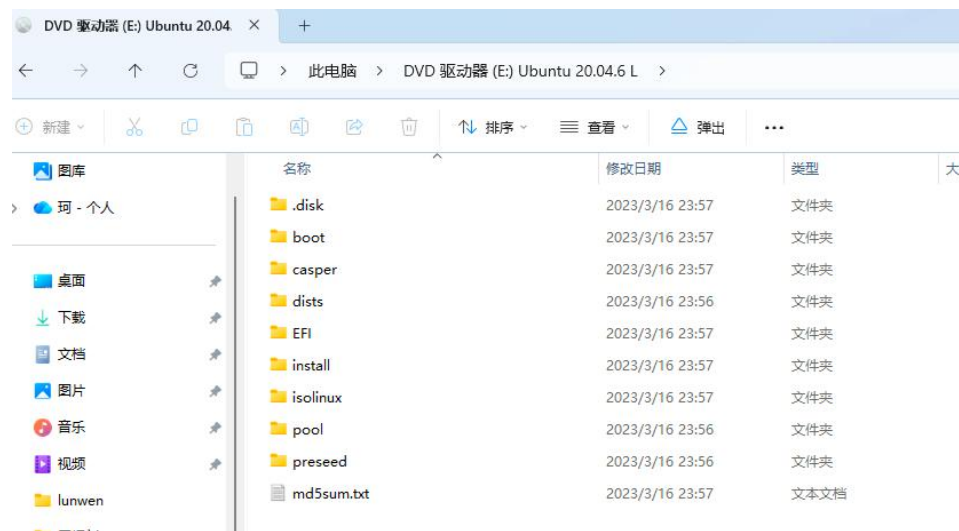
我们知道文件系统可以用来存储各种类型的文件：.img .txt .mp4等。

其实文件系统本身也可以变成一种文件：

文件系统镜像文

件：.iso .img(ext4, xfs, fat32) .raw等。

 ubuntu-20.04.6-desktop-amd64.iso



分区就是物理磁盘的**逻辑划分**，文件系统是对分区进行格式化后建立的**存储规则**。

1. 文件系统类型的文件

- 现代操作系统在启动时，会通过加载**磁盘驱动程序**（如 NVMe、AHCI 等）来识别并控制物理硬盘。
- 驱动程序通过访问硬件控制器寄存器获取磁盘信息，向内核注册**块设备**节点。
- 随后内核读取分区表（如 GPT），识别每个分区的文件系统类型（如 ext4、vfat、ntfs），
- 最终将这些分区**挂载**到文件系统目录树中，供操作系统和用户访问。

```
(base) xuke@lab230-p2:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
nvme1n1      259:0    0 953.9G  0 disk
├─nvme1n1p1 259:1    0   512M  0 part /boot/efi
└─nvme1n1p2 259:2    0 953.2G  0 part /
nvme2n1      259:3    0 238.5G  0 disk
├─nvme2n1p1 259:4    0   100M  0 part
├─nvme2n1p2 259:5    0    16M  0 part
├─nvme2n1p3 259:6    0 237.8G  0 part
└─nvme2n1p4 259:7    0   608M  0 part
nvme0n1      259:8    0   1.8T  0 disk
└─nvme0n1p1 259:9    0   1.8T  0 part /mnt/mydata
```

```
(base) xuke@lab230-p2:/boot$ sudo parted -l
```

型号: DAPUSTOR DPH312T4T001T9 (nvme)
磁盘 /dev/nvme0n1: 1920GB
扇区大小 (逻辑/物理): 512B/512B
分区表: gpt
磁盘标志:

编号	起始点	结束点	大小	文件系统	名称	标志
1	1049kB	1920GB	1920GB	ext4		

型号: SAMSUNG MZVLW256HEHP-000L7 (nvme)
磁盘 /dev/nvme2n1: 256GB
扇区大小 (逻辑/物理): 512B/512B
分区表: gpt
磁盘标志:

编号	起始点	结束点	大小	文件系统	名称	标志
1	1049kB	106MB	105MB	fat32	EFI system partition	启动, esp
2	106MB	123MB	16.8MB		Microsoft reserved partition	msftres
3	123MB	255GB	255GB	ntfs	Basic data partition	msftdata
4	255GB	256GB	638MB	ntfs		隐藏, 诊断

型号: INTEL SSDPEKNU010TZ (nvme)
磁盘 /dev/nvme1n1: 1024GB
扇区大小 (逻辑/物理): 512B/512B
分区表: gpt
磁盘标志:

编号	起始点	结束点	大小	文件系统	名称	标志
1	1049kB	538MB	537MB	fat32	EFI System Partition	启动, esp
2	538MB	1024GB	1023GB	ext4		

2. qemu接入块设备——磁盘

1、在 QEMU 中，一切“磁盘”都是一个文件。

本次实验qemu将我们mkfs创建的文件系统文件：fs.img当作物理磁盘“接入”到了qemu模拟的虚拟开发板virtio上。

```
QEMUOPTS += -drive file=$(TARGET_FS_PATH)/$(FS_IMG),if=none,format=raw,id=x0
QEMUOPTS += -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

2、QEMU 模拟的 VirtIO 设备是一个“固定地址的内存映射 I/O (MMIO) 设备。固定内存地址只能对应特定设备，不能冲突。

QEMU 在启动时就创建这些固定 MMIO 设备：

设备	MMIO 起始地址	说明
UART 串口	0x1000_0000	控制台输出设备
VirtIO 磁盘设备	0x1000_1000	虚拟块设备控制器
PLIC 中断控制器	0x0C00_0000	外部中断控制器
CLINT	0x0200_0000	计时器/核间中断
DRAM	0x8000_0000	主内存

3、对操作系统来说，除了 CPU 之外的几乎所有硬件都被视为“设备 (device)”，都需要驱动程序来初始化和控制。唯独内存 (RAM) 例外，因为它直接连接在 CPU 的内存总线上，CPU 可以通过 load / store 指令直接访问。内存的基本初始化 (检测、配置) 在启动时由 BIOS/UEFI 完成，随后由操作系统接管并进行内存管理。

4、linux中设备也是一种文件，在/dev下面，通过设备文件来让用户与设备交互，本次实验也需要初始化一个块设备文件disk来与接入的磁盘文件块设备io交互。

```
dd if=test_data.bin of=/dev/sda bs=4096 seek=100
```

```
xuke@xuke-virtual-machine:~/os-lab/ecnu-oslab-2025-answer$ cd /dev
xuke@xuke-virtual-machine:/dev$ ls
autofs      cpu_dma_latency  full          loop0         loop5         mqueue       rfkill       shm
block       cuse             fuse          loop1         loop6         net           rtc          snapshot
bsg         disk             hidraw0       loop10        loop7         null          rtc0         snd
btrfs-control dma_heap         hpet          loop11        loop8         nvram         sda          sr0
bus          dmide            hugepages     loop12        loop9         port          sda1         sr1
cdrom        dri              hwrng         loop13        loop-control  ppp           sda2         stderr
char         ecryptfs         initctl       loop14        mapper        psaux         sda3         stdin
console      fb0              input         loop2         mcelog        ptmx          sg0          stdout
core         fd               kmsg          loop3         mem           pts           sg1          tty
cpu          fd0              log           loop4         midi          random        sg2          tty0
```


3. 文件系统结构

我们实验采用类似ext4的文件系统格式——经典**inode**方案。

fs.img二进制布局（每个磁盘块1024字节）：

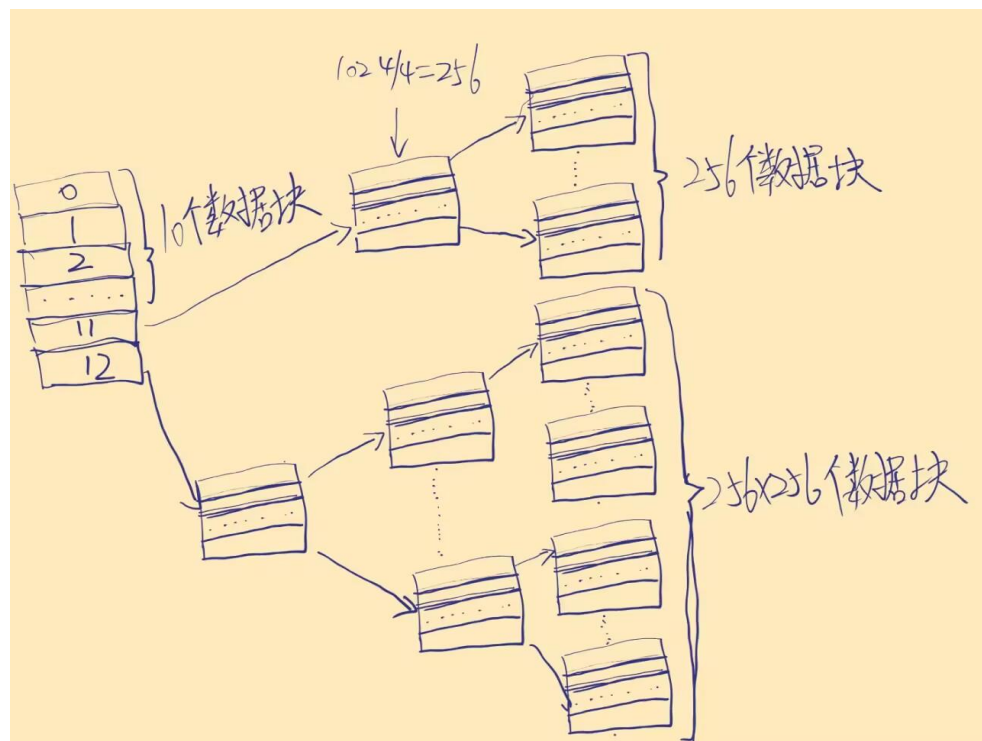
超级块	inode位图	128块存储inode节点	数据位图	8192块数据块
-----	---------	---------------	------	----------

inode节点：

```
// 一个inode 64字节大小
typedef struct inode_disk
{
    short type;           // 文件类型
    short major;          // 设备文件主设备号（针对设备文件）
    short minor;          // 设备文件次设备号（针对设备文件）
    short nlink;          // 硬链接数量
    unsigned int size;     // 文件大小（字节数）
    unsigned int addrs[13]; // 数据块地址表（指向存储文件内容的数据块）
} inode_disk_t;
```

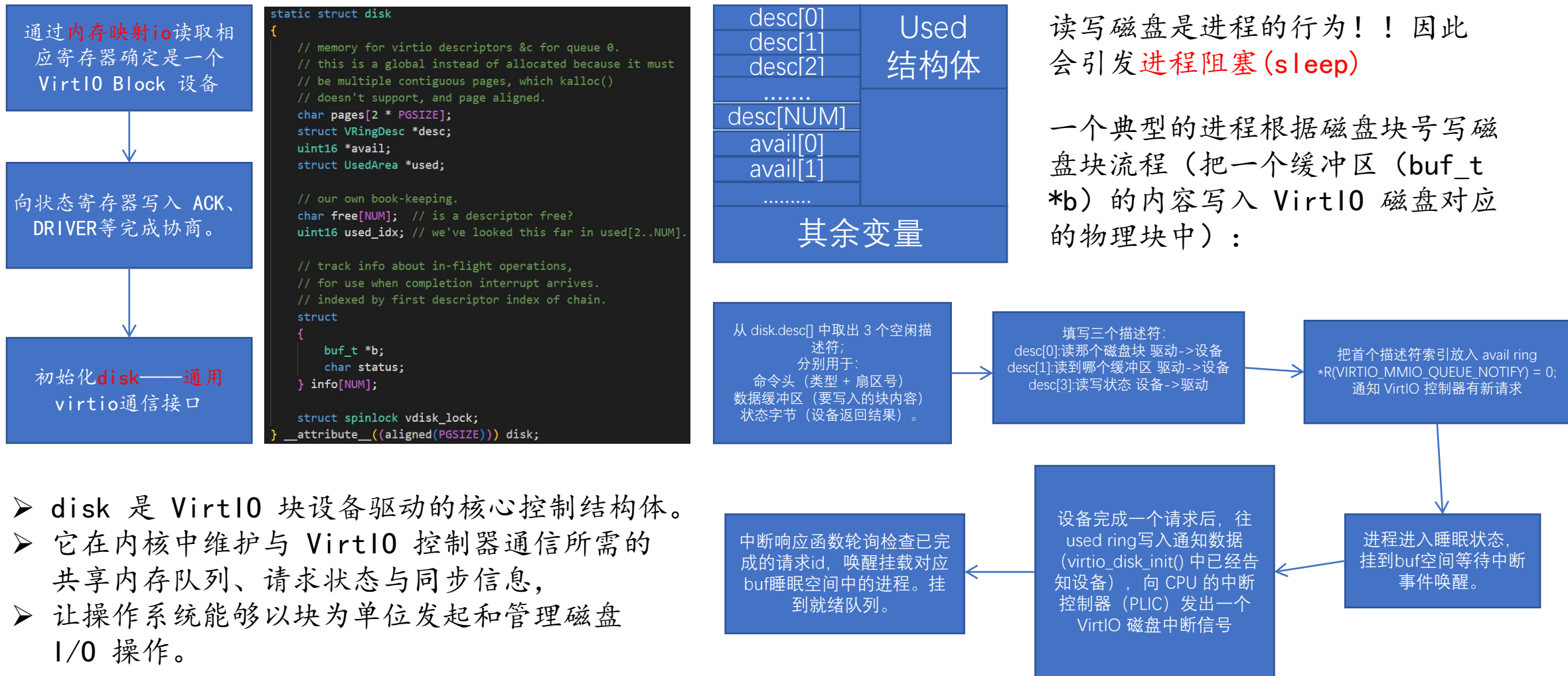
每个inode节点占64B，一个磁盘块能存16个inode，128个磁盘块存储2048个inode，也就是本文件系统最多支持2048个文件，inode位图只有前2048比特位有效。

inode能够管理的最大文件尺寸：



4. 文件系统读写

驱动程序初始化: virtio_disk_init()

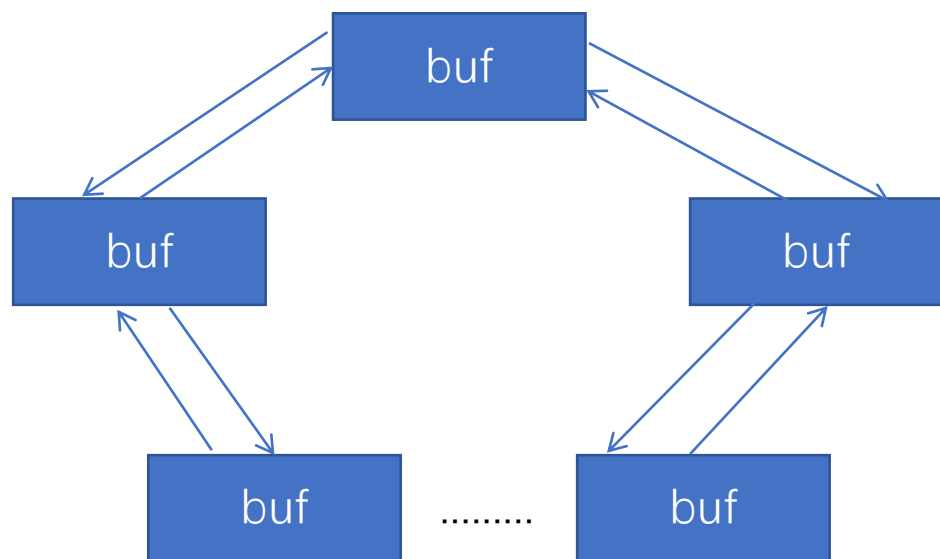


5. 内核VFS初始化

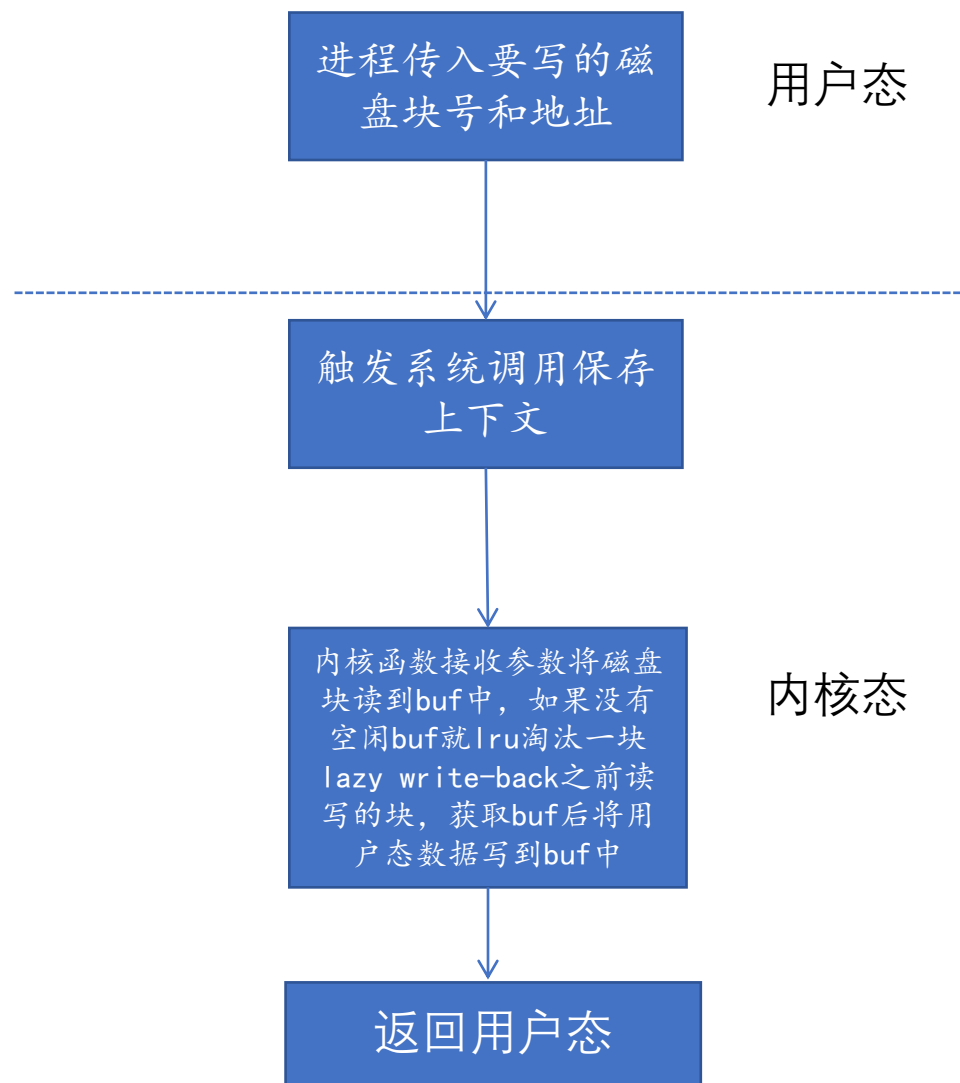
现代操作系统在启动的时候由**零号进程**初始化VFS文件系统，给下层各种类型的文件系统提供统一的管理接口来读写存储设备。

本次实验的内核fs_init()只针对fs.img中自定义的类ext4文件系统进程初始化。依然由零号进程在返回用户态。

本次实验主要实现内核当中的磁盘buf cache：块设备 I/O 的内存缓存，为文件系统访问磁盘时提供一个在内存中的数据块缓存池。



实现用户态磁盘块读写系统调用（以写磁盘块为例）：



6. 总结

- 我们介绍了现代文件系统在物理存储体上的分布，以及现代操作系统是通过VFS给底层异构的文件系统提供统一的api接口。
- 我们了解了现代操作系统能够将设备抽象成文件放到文件系统中通过设备文件来与设备交互。采用的是内存映射IO的方式对设备编写驱动程序与其通信。
- 我们通过文件的形式创建了一个类ext4格式的文件系统文件fs.img，并介绍了它的管理布局。
- 我们了解了qemu是通过指定文件的形式将我们创建的fs.img当作虚拟磁盘进行接入。
- 我们了解了在内核中如何根据virtio协议为外部磁盘设备编写初始化驱动程序的大致流程。
- 同时了解如何根据virtio协议来构造数据结构，并遵循协议流程来读写磁盘设备。
- 最后我们在内核中简单初始化了内核文件系统VFS，提供了外部磁盘块读写的buf cache区域并介绍了用户进程读写磁盘块的系统调用流程。